

Distributed Scene Update (DSU): A Middleware Pattern for Real-Time Partial State Synchronization

Robert Guy

Abstract

Many real-time, distributed applications maintain a shared “scene” or world state: a collection of entities and their attributes that must be updated and observed across multiple processes or services. Common examples include digital twins, simulation dashboards, monitoring visualizations, and online control systems. Naïve designs frequently rely on ad-hoc message formats and full-state broadcasts, which couple producers and consumers tightly, waste bandwidth, and make it difficult to reason about correctness when updates are delayed, duplicated, or reordered. This paper proposes *Distributed Scene Update (DSU)*, a middleware pattern and data model for partial, real-time state synchronization. DSU treats all application changes as a stream of declarative scene updates rather than imperative commands. Each update is a small, self-contained description of what should change in the global scene, together with identifiers and versioning metadata that enable merging, idempotence, and conflict resolution. We formalize the DSU model, provide merge semantics for applying updates to a canonical scene, and describe a transport-agnostic middleware architecture (e.g., compatible with publish/subscribe messaging or RPC). We then analyze correctness properties such as convergence under at-least-once delivery and discuss performance trade-offs in update granularity, batching, and indexing. Finally, we outline a reference implementation and an evaluation plan based on a real-time multi-service simulation dashboard. We illustrate the model using a running example involving a simulated vehicle entity. DSU aims to provide a reusable abstraction: developers focus on what needs to change in the scene, while the middleware ensures consistent, efficient propagation of those changes across a distributed system.

1 Introduction

Many modern systems maintain a shared, structured view of the world. Examples include real-time simulation dashboards, collaborative visualization tools, digital twins for physical environments, and monitoring systems which aggregate events from heterogeneous sources. We refer to this shared world representation as the *scene*. A common pattern in practice is to handcraft message formats for each application and push either:

- Full snapshots of scene state or
- Imperative commands (“add object x ”, “move object y ”).

These approaches are simple to implement but have important drawbacks:

1. **Bandwidth and scaling.** Full snapshots become expensive as the scene grows; sending them at interactive rates quickly becomes infeasible.

2. **Tight coupling.** Imperative command sets are often application-specific and strongly coupled to internal data structures. Adding a new type of entity or attribute requires changing many components.
3. **Distributed correctness.** Real deployments rarely enjoy perfect networking. Messages may be delayed, duplicated, or arrive out of order. Without a clear model, it is hard to guarantee that different subscribers converge to the same view of the scene.

The central insight behind Distributed Scene Update (DSU) is that scene synchronization can be expressed as a stream of *versioned partial state updates*. Instead of broadcasting full scene snapshots or replaying imperative commands, DSU propagates small declarative descriptions of what has changed in the scene. Each update carries identifiers and version metadata that allow replicas to deterministically merge updates even when messages are delayed, duplicated, or reordered. This approach preserves the simplicity of state updates while enabling efficient and robust distributed synchronization.

This paper introduces the *Distributed Scene Update (DSU)* pattern, a general-purpose abstraction for real-time scene synchronization:

- The scene is modeled as a set of typed entities with attributes.
- Producers emit *partial updates* that describe the desired state of some subset of entities.
- Each update carries identifiers and version metadata to allow merging, deduplication, and conflict resolution.
- Middleware maintains a canonical scene and propagates derived changes to subscribers via a transport-agnostic interface.

Our contributions are:

- A formalization of scenes, entities, and DSU update messages.
- Merge semantics for applying updates under at-least-once delivery, including idempotence and eventual convergence conditions.
- A reference middleware architecture that separates concerns between producers, the scene coordinator, and subscribers.
- A discussion of performance trade-offs, version allocation strategies, and an evaluation plan for real-time workloads.

DSU is intentionally domain-agnostic. Although many motivating examples come from signal-processing-heavy or time-sensitive systems, the pattern is equally applicable to any application that maintains a shared, structured view of the world state.

1.1 Positioning Relative to Existing Models

Distributed Scene Update occupies a design space between several existing approaches for distributed state management.

Conflict-Free Replicated Data Types (CRDTs) provide convergence by restricting updates to commutative operations. DSU instead introduces explicit per-entity version ordering, allowing deterministic resolution of partial updates without requiring commutativity of attribute updates.

State machine replication systems replicate deterministic commands that must be executed across all replicas. DSU differs by propagating declarative state updates rather than procedural commands, allowing subscribers to reconstruct state directly without replaying operations. Event sourcing architectures represent system state as an append-only log of events that must be replayed to reconstruct the current state. DSU instead transmits partial entity state updates directly, reducing replay overhead and simplifying subscriber implementations. Finally, publish/subscribe messaging infrastructures provide a transport mechanism but typically lack semantic merge logic. DSU introduces explicit versioning and merge semantics to guarantee deterministic convergence under duplication and reordering.

1.2 Running Example

Throughout this paper we use a simple running example to illustrate the DSU model. Consider a real-time simulation dashboard that tracks entities in a shared scene. Each entity represents an object in the simulation (e.g., a vehicle, sensor platform, or target). Producers emit updates when attributes such as position, velocity, or status change. Subscribers such as visualization dashboards or analytics services consume these updates to maintain a consistent view of the scene.

As a concrete example, suppose **Vehicle42** is represented by the attributes **position**, **velocity**, and **status**. A producer may first emit an update setting:

- **position** = (10,4)
- **velocity** = 5
- **status** = active

A later update may modify only **position** while leaving other attributes unchanged. Under DSU, subscribers receive only the relevant delta together with version metadata. If an older update for **Vehicle42** arrives later, it is discarded as stale. This example captures the core DSU use case: multiple producers generate incremental state updates for structured entities while multiple subscribers maintain derived views of the scene.

2 Problem Setting and Requirements

2.1 Scene model

We model the global scene at time t as a finite mapping

$$S_t : \mathcal{E} \rightarrow \mathcal{V}$$

where $\mathcal{E}$ is the space of entity identifiers and $\mathcal{V}$ captures per-entity data. For concreteness, we can treat each entity value $v \in \mathcal{V}$ as a record of attributes:

$$v = \{(k, a_k) \mid k \in K\}$$

where K is the set of attribute keys (e.g., position, status, label, etc.).

We assume:

- Entity IDs are globally unique within a scene instance.
- Attribute keys are known to producers and consumers (via a schema).
- Each entity has a *version* $ver(e)$, monotonically increasing with respect to updates.

2.2 Update sources and consumers

We consider three logical roles:

- **Producers** generate updates based on sensors, simulations, user input, or other services.
- **Coordinator** maintains the canonical scene S_t and applies updates according to DSU semantics.
- **Subscribers** consume changes to render visualizations, compute derived analytics, or drive downstream components.

These roles may be colocated in practice, but we separate them to clarify responsibilities.

2.3 Requirements

We target systems with the following requirements:

1. **Low latency.** Updates should propagate quickly enough for interactive use (e.g., sub-second response).
2. **Partial state updates.** Most changes affect a small subset of the scene; we should avoid resending the entire state.
3. **At-least-once delivery.** Middleware may deliver updates more than once; the model should handle deduplication gracefully.
4. **Reordering tolerance.** Updates may arrive out of order due to network variability.
5. **Extensibility.** New entity types and attributes should not require redesigning the transport or coordination logic.

3 The Distributed Scene Update (DSU) Model

3.1 DSU update messages

A DSU update is a self-contained, declarative description of changes to the scene. Informally, an update says, “after you apply this message, these entities should look like this.”

We define an update U as:

$$U = (\text{id}, T, ts, \{u_1, \dots, u_m\})$$

where:

- id is a unique update identifier.
- T is an optional logical *topic* or scene namespace.
- ts is a timestamp or sequence number (logical or physical).
- Each u_i is an *entity delta*.

An entity delta has the form:

$$u_i = (e_i, \text{ver}_i, \text{op}_i, \Delta_i)$$

where:

- $e_i \in \mathcal{E}$ is the target entity identifier.
- $ver_i \in \mathbb{N}$ is the proposed new version for entity e_i .
- $op_i \in \{\text{upsert}, \text{delete}\}$ is the operation.
- Δ_i is a partial map of attribute changes: $\Delta_i : K \rightarrow \mathcal{A}$, where $\mathcal{A}$ is the attribute value space.

3.2 Merge semantics

Intuitively, DSU resolves conflicting updates by favoring the most recent version of an entity’s state. Updates that match the current version are treated as duplicates, while older updates are safely ignored.

Let S be the current canonical scene. Applying a single entity delta u_i produces a new scene S' using the following rules:

1. If $op_i = \text{delete}$: remove e_i if and only if $ver_i \geq ver(e_i)$; otherwise, ignore the delete as stale.
2. If $op_i = \text{upsert}$:
 - If e_i does not exist, insert it with version ver_i and attributes Δ_i (or a default base value extended by Δ_i).
 - If e_i exists with a version $ver(e_i)$:
 - If $ver_i > ver(e_i)$, treat u_i as the new source of truth: overwrite or merge attributes according to Δ_i , and set $ver(e_i) \leftarrow ver_i$.
 - If $ver_i = ver(e_i)$, apply Δ_i as a *commutative merge* (e.g., last-writer-wins per attribute, or attribute-specific policies). This ensures idempotence when the same update is delivered multiple times.
 - If $ver_i < ver(e_i)$, ignore the delta as stale.

Condition	Action
$ver_i > ver(e)$	Accept update and overwrite/merge state
$ver_i = ver(e)$	Idempotent merge (duplicate update)
$ver_i < ver(e)$	Drop update as stale

Table 1: DSU merge semantics based on version comparison.

In the running example, if an update for **Vehicle42** with version 8 has already been applied and a delayed version 7 update later arrives, the version 7 update is discarded. If the same version 8 update is delivered twice, the second delivery does not change the final scene state.

Algorithm 1 sketches the core logic.

Algorithm 1 Apply a DSU update U to scene S

Require: Scene S , update $U = (\text{id}, T, ts, \{u_1, \dots, u_m\})$

```
1: for all  $u_i = (e_i, ver_i, op_i, \Delta_i)$  in  $U$  do
2:    $v \leftarrow S[e_i]$  (or  $\perp$  if not present)
3:   if  $op_i = \text{delete}$  then
4:     if  $v \neq \perp$  and  $ver_i \geq ver(v)$  then
5:       remove  $e_i$  from  $S$ 
6:   else
7:     {upsert}
8:     if  $v = \perp$  then
9:       create  $v$  with default attributes
10:       $ver(v) \leftarrow ver_i$ 
11:   else
12:     if  $ver_i < ver(v)$  then
13:       continue {stale; ignore}
14:     else if  $ver_i > ver(v)$  then
15:        $ver(v) \leftarrow ver_i$ 
16:       apply all attributes in  $\Delta_i$  to  $v$ 
17:     else
18:       apply attributes in  $\Delta_i$  using per-attribute merge policy (e.g., last-writer-wins)
```

3.3 Convergence under at-least-once delivery

Assume:

- Each entity's updates are totally ordered by ver_i at their source.
- Each update is eventually delivered to the coordinator at least once.
- The merge policy for equal versions is commutative and idempotent.

Then, despite duplication and reordering, the coordinator's scene converges to a well-defined limit as all updates are applied. Intuitively, each entity tracks the greatest version it has seen; lower-version updates are discarded, and reapplying an update with the same version does not change the result.

4 Design Rationale

The DSU model makes several deliberate design choices intended to balance simplicity, correctness, and performance in real-time distributed systems.

Versioned entity updates. DSU assigns monotonically increasing versions to individual entities rather than relying on global ordering. This allows independent producers to update different entities concurrently while preserving a clear ordering of updates for each entity.

Declarative state updates rather than commands. Unlike command-based replication models where operations must be replayed deterministically, DSU updates describe the desired state of entities. This avoids tight coupling between producers and consumers.

Partial state propagation. Many real-time systems update only a small subset of attributes at high frequency. DSU therefore represents updates as attribute deltas rather than full entity snapshots.

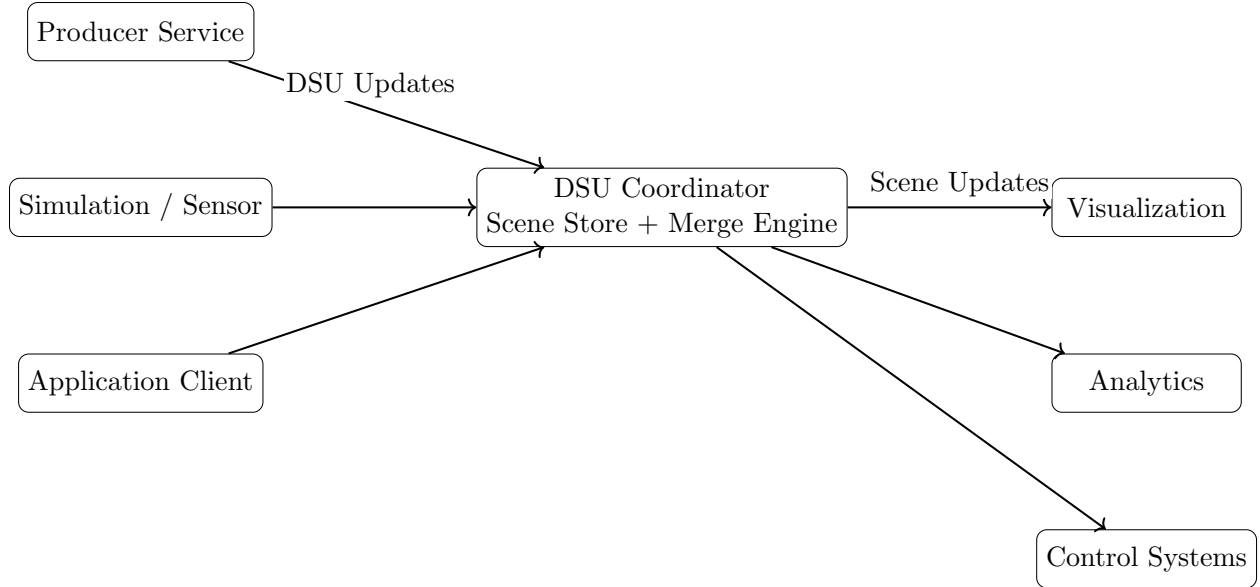


Figure 1: DSU middleware architecture. Producers emit scene updates which are transported via a messaging layer to subscribers. The design is transport-agnostic and compatible with publish/subscribe or RPC systems.

Transport independence. The DSU abstraction intentionally avoids dependence on a specific messaging system, allowing implementations over RPC, pub/sub messaging, or log-based streaming systems.

5 DSU Middleware Architecture

5.1 High-level components

A DSU middleware deployment typically includes:

- **Client SDK.** A library that applications use to construct DSU updates in a strongly-typed, schema-driven way (e.g., a code generator that produces classes from a schema definition).
- **Coordinator service.** Maintains the canonical scene, applies updates, and emits derived change events to interested subscribers.
- **Transport layer.** Any reliable-enough messaging substrate (e.g., publish/subscribe middleware or RPC) used to deliver updates and broadcast changes.
- **Subscriber SDK.** Provides APIs to subscribe to full scene snapshots, incremental state diffs, or filtered views (e.g., entities matching a predicate).

The key idea is that application code never manipulates the scene by directly mutating shared in-memory structures in other services. Instead, it always sends DSU updates through the middleware, which becomes the single source of truth.

5.2 Transport-agnostic design

DSU intentionally does not prescribe a transport implementation. Updates are just messages; they can be serialized using binary formats (e.g., a schema-based encoder) or JSON, and carried by:

- message brokers (e.g., pub/sub systems),
- direct RPC calls (e.g., a coordinator endpoint),
- or even log-based systems (e.g., an append-only event log).

The only transport-level assumptions are:

1. Messages are eventually delivered, possibly with duplication.
2. There is some mechanism to partition topics or namespaces (e.g., by scene or event).

5.3 Version Allocation Strategies

Strategy	Advantage	Trade-off
Per-producer ranges	No runtime coordination	Static allocation overhead
Composite versions (timestamp, producer)	Decentralized generation	Dependent on clock synchronization
Hybrid Logical Clocks (HLC)	Monotonic ordering with bounded skew	Slightly more complex implementation
Centralized allocation	Strong ordering guarantees	Coordinator throughput bottleneck

Table 2: Comparison of DSU version allocation strategies.

A practical DSU deployment must determine how update versions are generated across producers while preserving ordering guarantees.

Per-producer ranges. Each producer is assigned a disjoint range of version identifiers. This avoids coordination but requires pre-allocation and may waste identifier space if producers emit updates at different rates.

Composite versions. Versions may be represented as tuples such as $(timestamp, producer_id)$. This allows independent producers to generate versions without direct coordination, while preserving a deterministic order when timestamps are sufficiently synchronized.

Hybrid logical clocks. Hybrid logical clocks (HLCs) combine physical timestamps with logical counters to produce monotonically increasing versions while tolerating bounded clock skew.

Centralized allocation. In some deployments the coordinator may allocate version identifiers directly. This simplifies ordering guarantees but introduces a coordination point that may limit throughput.

The best strategy depends on deployment requirements, expected update rates, the number of producers, and the acceptable coordination overhead. In a multi-writer setting, strategies with stronger ordering guarantees reduce the likelihood of conflicting concurrent updates but may incur higher coordination costs.

5.4 Multi-coordinator deployments

Large-scale deployments may require multiple coordinators for fault tolerance or geographic distribution. In such scenarios, coordinators can replicate the canonical scene using a consensus protocol such as Raft or Paxos.

Under this design, DSU updates are appended to a replicated log and applied deterministically across coordinator replicas. Alternatively, the scene may be partitioned by an entity identifier, with shards responsible for subsets of the global scene. Cross-shard updates are routed to the appropriate shard based on the target entity identifier.

Consistency model. Under a single-coordinator deployment, DSU preserves a globally ordered update stream, providing behavior similar to state-machine replication: all subscribers observe updates in the same version order. In multi-coordinator deployments, updates may arrive in different orders at different replicas. However, deterministic version-based merge semantics ensure eventual convergence. Once the highest version update for each entity has been received, all replicas converge to the same scene state. This allows DSU deployments to support both strongly ordered and eventually consistent synchronization modes depending on the coordinator topology.

These approaches preserve the DSU abstraction while allowing the system to scale beyond a single coordinator process.

5.5 Subscriptions and derived views

Subscribers often do not need the entire scene; they may render only a region, entity type, or aggregated view. The DSU coordinator can therefore expose different subscription APIs, such as:

- `SubscribeFull()`: delivers periodic full snapshots plus incremental updates.
- `SubscribeRegion(filter)`: delivers updates for entities that match a spatial or logical filter.
- `SubscribeChanges(since_t)`: delivers a replay of updates from a given timestamp or version.

Internally, the coordinator may maintain indices (by ID, by type, by region) to efficiently support these queries.

6 Correctness Properties

We now discuss the main correctness properties DSU aims to provide. We assume a single coordinator for simplicity; multi-coordinator deployments can be handled with standard state machine replication techniques.

6.1 Idempotence

An update U is *idempotent* if applying it multiple times has the same effect as applying it once:

$$\text{Apply}(S, U, U) = \text{Apply}(S, U)$$

where `Apply` denotes the deterministic result of applying updates in sequence.

Under the versioning and merge rules in Algorithm 1, idempotence follows because:

- Once an entity reaches version ver_i , subsequent applications of the same update with the same ver_i either:
 - are ignored as stale (if we store that we have already processed id), or
 - merge attributes using an idempotent rule (e.g., last-writer-wins).

For example, if multiple services emit updates for `Vehicle42`, replicas may receive these updates in different orders. Because each update includes a version number and deterministic merge rule, all replicas eventually converge to the same final state once the highest version update has been applied.

6.2 Convergence

We say the system is *convergent* if all subscribers observing the same update stream (in any order, with duplication) eventually reach equivalent views of the scene. When all updates are applied at the coordinator according to DSU rules, convergence at the coordinator is guaranteed as argued earlier. Subscribers that consume derived deltas also converge because they are driven by the coordinator’s state.

6.3 Monotonicity of versions

Per-entity versions enforce a partial order on updates. If all producers respect monotonic version increments for each entity, then:

- The coordinator never rolls back an entity to an older version.
- Conflicts between concurrent producers can be resolved by choosing a versioning strategy (e.g., assigning ranges per producer or using a composite version such as $(clock, producer_id)$).

A production implementation would include additional details on version allocation strategies and conflict-resolution policies.

7 Failure Scenarios and Recovery

Distributed deployments inevitably experience component failures and intermittent network disruptions. DSU is designed to tolerate these scenarios through versioned, idempotent updates.

7.1 Coordinator restart

If the coordinator restarts, the canonical scene can be reconstructed from the persistent update log. Because updates are declarative and versioned, replaying them reconstructs the same scene state.

7.2 Producer reconnection

Producers may temporarily disconnect and later reconnect. Upon reconnection they resume sending updates with higher versions. Older versions are ignored by the coordinator.

7.3 Subscriber recovery

Subscribers that disconnect may request either a full snapshot or a replay of updates beginning from a known version.

Consider a network delay where version 5 of `Vehicle42` arrives before version 4. The DSU merge rule accepts version 5 and records it as the latest state. When version 4 eventually arrives, it is detected as stale and safely discarded.

8 Performance Considerations

Although DSU is conceptually simple, real systems must balance flexibility with performance.

8.1 Update granularity

Choosing update granularity involves a trade-off:

- **Fine-grained updates** (small Δ_i) reduce redundant data transfer but increase message overhead and coordination work.
- **Coarse-grained updates** (entire entity snapshots) simplify merge logic but may resend many unchanged attributes.

A practical design is to support multiple levels:

- high-frequency, fine-grained updates for rapidly changing attributes (e.g., positions, counters),
- lower-frequency snapshots for slowly changing attributes (e.g., labels, configuration).

8.2 Batching and indexing

To improve throughput:

- Batch multiple entity deltas into a single update U when possible.
- Maintain indices for common queries (by entity ID, by type, by region) to avoid scanning the entire scene for every subscription.
- Use write-optimized data structures (e.g., append-only logs plus periodic compaction) when update rates are high.

Batching can also mitigate cache-line contention on multi-core systems by processing groups of updates in worker threads rather than a single, fine-grained lock per entity.

9 Reference Implementation Sketch

Although this paper focuses on the model, we briefly outline a reference implementation to clarify the APIs.

9.1 Schema and code generation

Define the scene schema in a language-agnostic format, for example:

- entity types (e.g., `Node`, `Link`, `Metric`),
- attribute keys and types per entity,
- version fields and identifiers.

A code generator then produces:

- language-specific classes or structs for entities and updates,
- serialization and deserialization routines,
- client-side helper methods such as `upsertEntity`, `deleteEntity`, etc.

9.2 Coordinator service

The coordinator maintains:

- an in-memory scene store (e.g., hash map from ID to entity struct),
- a persistent log of incoming updates (for replay and debugging),
- subscription tables for downstream clients.

When an update arrives:

1. Deserialize it and validate basic invariants.
2. Apply it to the scene using Algorithm 1.
3. Compute derived changes (e.g., which entities changed, which subscribers care).
4. Forward deltas to subscribers.

9.3 Client SDK example

A client library might expose an API like:

```
SceneClient client(config);

EntityId id = ...;
SceneDelta delta;
delta.setPosition(id, newPos);
delta.setStatus(id, STATUS_ACTIVE);
client.sendUpdate(delta);
```

Under the hood, this constructs a DSU update U with appropriate versions and sends it to the coordinator via the configured transport.

10 Example Applications

DSU is broadly applicable to systems that maintain structured world state.

10.1 Real-time simulation dashboards

Simulation environments frequently maintain hundreds or thousands of entities whose attributes evolve over time. DSU enables efficient propagation of incremental updates.

10.2 Digital twin systems

Digital twin architectures replicate physical environments in software. Multiple producers update the shared model while monitoring systems subscribe to derived state.

10.3 Collaborative visualization

Multi-user visualization tools maintain shared scenes containing objects, annotations, and measurements. DSU synchronizes these scenes efficiently.

11 Evaluation Plan

Although this paper focuses primarily on the DSU model and middleware design, we outline an evaluation methodology for assessing the performance and scalability of DSU in real-time distributed systems.

In the evaluation workload, we simulate a scene containing up to 10,000 entities (e.g., vehicles such as `Vehicle42`) whose attributes are updated at configurable rates.

11.1 Experimental setup

- Synthetic workload generator that produces entity updates at configurable rates, with controllable skew (e.g., many small updates vs. fewer larger ones).
- Number of entities: 1k–10k
- Update rate: up to 1k updates/sec
- Network latency: 5–50 ms
- Message duplication: 1–5%
- Baseline implementations:
 - full-state broadcast (snapshot every T ms),
 - ad-hoc command-based update scheme without DSU semantics,
 - an event-sourcing style replay pipeline,
 - a CRDT-based state propagation scheme for comparable entity types.
- DSU implementation using a commodity messaging substrate.

11.2 Metrics

- **Bandwidth usage** (bytes per second).
- **Update latency** (time from producer emission to subscriber observation).
- **CPU utilization** at the coordinator.
- **Consistency error**: fraction of time that subscribers disagree on entity states under injected delays, duplication, and reordering.
- **Scalability**: throughput as the number of entities, subscribers, and producers increases.

11.3 Hypotheses

We expect DSU to:

- substantially reduce bandwidth compared to full-state broadcast, especially for sparse updates,
- provide cleaner convergence guarantees than ad-hoc command schemes, with modest overhead from versioning and merge logic,
- outperform replay-based approaches when subscribers only need the current state rather than a full event history,
- exhibit predictable behavior under degraded network conditions due to idempotence and explicit versioning.

12 Glossary

- **Scene**: The global structured state shared across distributed components.
- **Entity**: An individual object in the scene identified by a unique identifier.
- **Delta**: A partial update that modifies some subset of an entity's attributes.
- **Version**: Metadata attached to updates that determines ordering and conflict resolution.

13 Limitations and Future Work

DSU focuses on a single-coordinator model; scaling to geographically distributed coordinators requires additional mechanisms (e.g., consensus protocols or CRDT-style replication).

Conflict resolution between multiple writers updating the same entity concurrently is also application-dependent and may require domain-specific policies.

Future work includes:

- Formalizing DSU as a class of CRDTs for common attribute types.
- Integrating with existing streaming engines (e.g., dataflow systems) to provide higher-level declarative queries.

- Providing language bindings and tooling for automatic schema evolution and compatibility checking.
- Evaluating DSU under multi-coordinator and sharded deployments.
- Formal verification of convergence properties and large-scale deployment studies.

14 Related Work and Context in Distributed Systems

DSU sits at the intersection of several well-studied areas in distributed systems. Classical work on logical time such as Lamport clocks [1] and vector clocks [2] provides mechanisms for ordering events in distributed systems. Replicated state machine approaches [5] and consensus protocols such as Paxos [6] and Raft [7] provide strong consistency through deterministic command replication.

In contrast, DSU focuses on partial state synchronization where only subsets of entity attributes change over time. This places DSU closer to replicated data structure approaches such as CRDTs [3] and delta-state CRDTs [4]. However, DSU introduces explicit entity version ordering rather than relying purely on commutative merge functions.

Event-based system designs such as event sourcing [8] represent state as a log of events. DSU differs by transmitting partial entity state rather than replaying procedural commands. Similarly, publish/subscribe messaging infrastructures [9] are commonly used to disseminate state updates. Recent work on digital twin architectures [10] highlights the need for efficient entity-level synchronization mechanisms for large-scale simulated environments.

15 Conclusion

This paper introduced *Distributed Scene Update (DSU)*, a middleware pattern for synchronizing structured scene state in distributed systems using versioned partial updates. By representing application changes as declarative deltas rather than imperative commands or full snapshots, DSU enables efficient propagation of updates while preserving deterministic convergence under delayed, duplicated, or reordered message delivery.

Our contributions include:

- the DSU model for partial scene updates
- deterministic merge semantics based on version ordering
- a transport-agnostic middleware architecture
- version allocation strategies for distributed producers
- extensions for multi-coordinator deployments
- a concrete evaluation plan comparing DSU with CRDT and event-sourced approaches

DSU provides a practical abstraction for real-time distributed systems in which scene state must be updated efficiently while remaining robust to network imperfections such as delay, duplication, and reordering.

References

- [1] Lamport, L. Time, Clocks, and the Ordering of Events in a Distributed System. Communications of the ACM, 1978.
- [2] Fidge, C. Timestamps in Message-Passing Systems. 1988.
- [3] Shapiro, M., Preguiça, N., Baquero, C., Zawirski, M. Conflict-Free Replicated Data Types. SSS, 2011.
- [4] Almeida, P. et al. Delta State Replicated Data Types. Journal of Parallel and Distributed Computing, 2018.
- [5] Schneider, F. Implementing Fault-Tolerant Services Using the State Machine Approach. ACM Computing Surveys, 1990.
- [6] Lamport, L. Paxos Made Simple. ACM SIGACT News, 2001.
- [7] Ongaro, D. and Ousterhout, J. In Search of an Understandable Consensus Algorithm (Raft). USENIX ATC, 2014.
- [8] Fowler, M. Event Sourcing. 2005.
- [9] Eugster, P. et al. The Many Faces of Publish/Subscribe. ACM Computing Surveys, 2003.
- [10] Tao, F. et al. Digital Twin Driven Smart Manufacturing. IEEE Access, 2018.